Pre-Req

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def show_images(images, titles, columns=3):
    plt.figure(figsize=(20, 10))
    for i, (img, title) in enumerate(zip(images, titles)):
        plt.subplot(len(images) // columns + 1, columns, i + 1)
        if len(img.shape) == 2:
            plt.imshow(img, cmap='gray')
        else:
            plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        plt.title(title)
        plt.axis('off')
    plt.show()
```

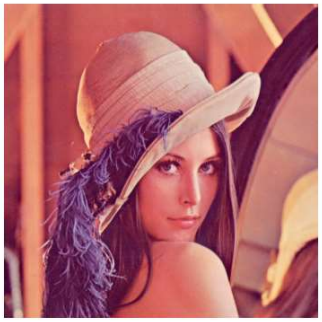
```
lena = cv2.imread('Lena_Image.jpg')
face = cv2.imread('face.jpg')
medical = cv2.imread('medical2.jpeg', 0)
lena_gray = cv2.imread('Lena_grey.jpg', 0)
map_img = cv2.imread('map.jpg', 0)
bag = cv2.imread('bag.jpg', 0)
butterfly = cv2.imread('Butterfly.png', 0)
pyramid = cv2.imread('pyramid.webp')
```

Color Model Conversion

```
def convert_colors(img, name):
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    ycbcr = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)

    bgr_norm = img.astype(float) / 255.0
    k = 1 - np.max(bgr_norm, axis=2)
    c = (1 - bgr_norm[:,:,2] - k) / (1 - k + 1e-7)
    m = (1 - bgr_norm[:,:,1] - k) / (1 - k + 1e-7)
    y = (1 - bgr_norm[:,:,0] - k) / (1 - k + 1e-7)

    cmyk = (np.dstack((c, m, y, k)) * 255).astype(np.uint8)
    cmy = (np.dstack((1-bgr_norm[:,:,2], 1-bgr_norm[:,:,1], 1-bgr_norm[:,:,0])) * 255).astype(np.uint8)

    show_images([img, hsv, ycbcr, cmy], [f'{name} RGB', 'HSV', 'YCbCr', 'CMY'])
```

```
convert_colors(lena, "Lena")
convert_colors(face, "Face")
```

Lena RGB



HSV

YCbCr

CMY

Face RGB

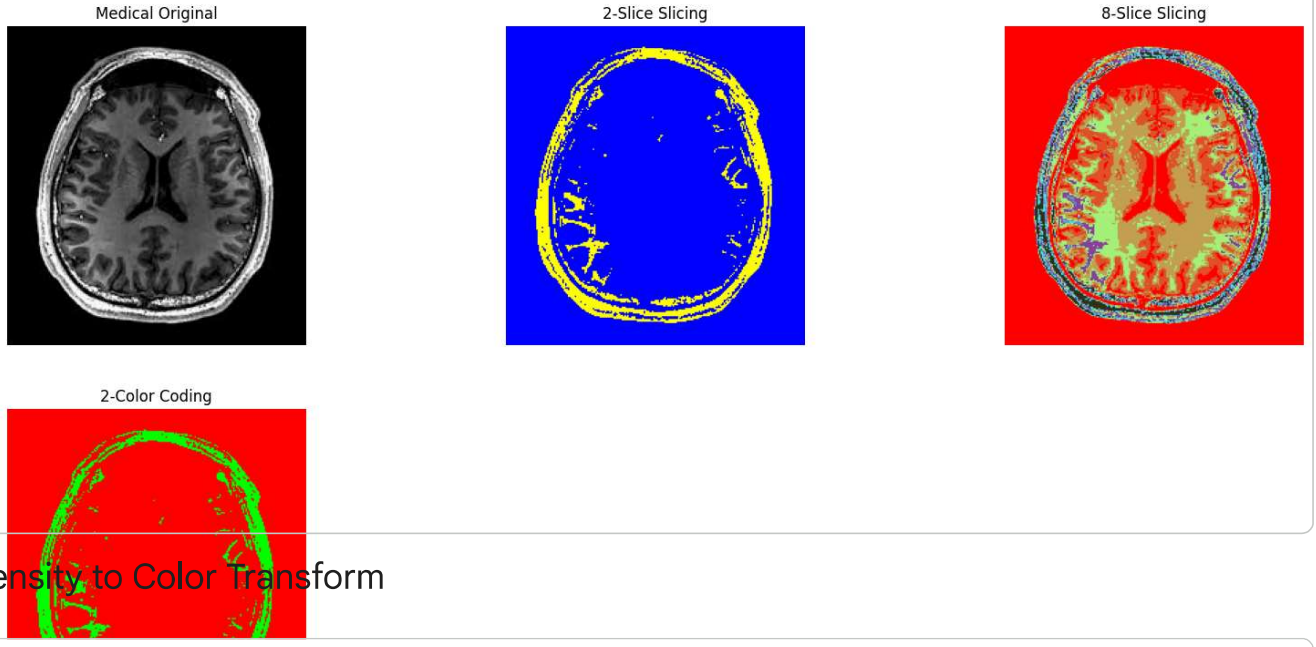
HSV

YCbCr

CMY

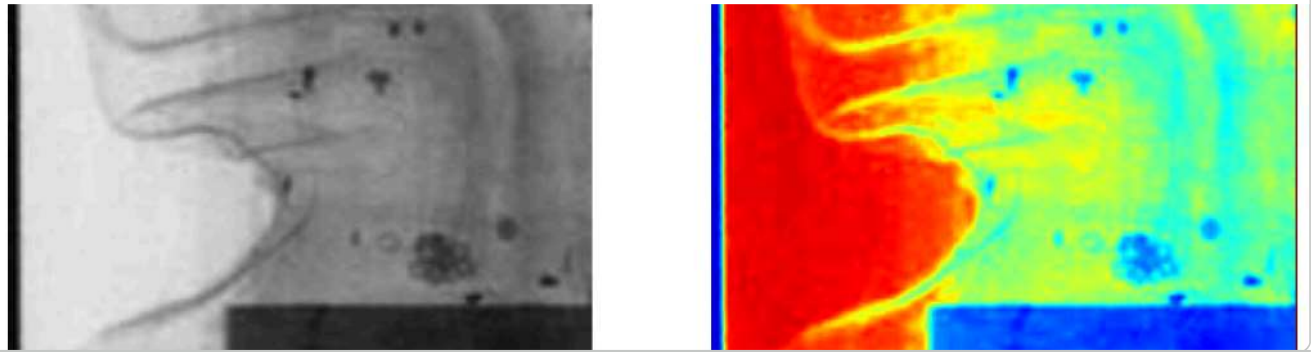
Pseudo Color Image Processing

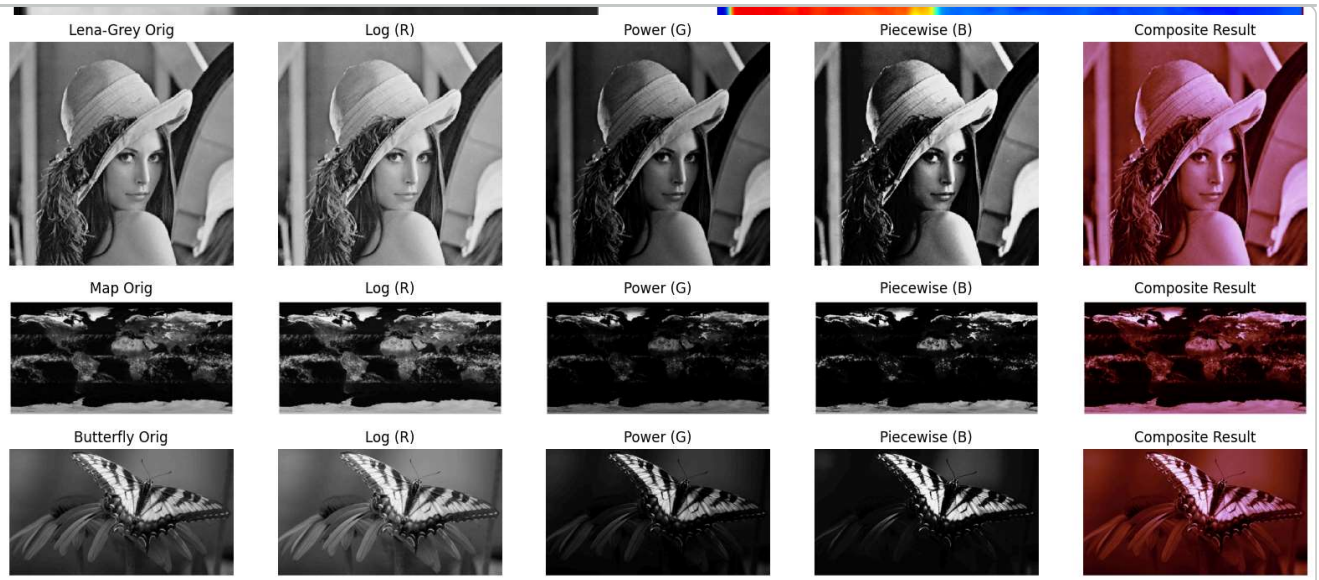
```
def q2_pseudo_color_revisited(medical_img):
    # 2-slice Intensity Slicing (Threshold at 128)
    h, w = medical_img.shape
    slice2 = np.zeros((h, w, 3), dtype=np.uint8)
    slice2[medical_img < 128] = [255, 0, 0]
    slice2[medical_img >= 128] = [0, 255, 255]

    # 8-slice Intensity Slicing
    slice8 = np.zeros((h, w, 3), dtype=np.uint8)
    for i in range(8):
        lower, upper = i * 32, (i + 1) * 32
        mask = cv2.inRange(medical_img, lower, upper - 1)
        color = [ (i*40)%255, (i*80)%255, 255 - (i*30)%255 ]
        slice8[mask > 0] = color

    # 2-color only coding (Specific Binary Mapping)
    two_color = np.zeros((h, w, 3), dtype=np.uint8)
    two_color[medical_img < 128] = [0, 0, 255] # Red (BGR)
    two_color[medical_img >= 128] = [0, 255, 0] # Green (BGR)

    show_images([medical_img, slice2, slice8, two_color],
                ['Medical Original', '2-Slice Slicing', '8-Slice Slicing', '2-Color Coding'])

    # Multi-color intensity coding for other images
    gray_list = [lena_gray, map_img, bag]
    names = ["Lena-Grey", "Map", "Bag"]

    for g_img, name in zip(gray_list, names):
        if g_img is not None:
            mcc = cv2.applyColorMap(g_img, cv2.COLORMAP_JET)
            show_images([g_img, mcc], [f'Gray {name}', f'Multi-Color {name}'])

q2_pseudo_color_revisited(medical)
```



Intensity to Color Transform

```
def Intensity_to_color(img_gray, name):
    img_float = img_gray.astype(np.float32) / 255.0

    # 1. Log Transformation (Highlights darker regions)
    c = 1 / np.log(1 + 1)
    log_trans = c * np.log(1 + img_float)
    log_trans = np.uint8(log_trans * 255)

    # 2. Power-Law (Gamma) Transformation (Adjusts contrast)
    # Using gamma > 1
    gamma = 2.0
    power_trans = np.power(img_float, gamma)
    power_trans = np.uint8(power_trans * 255)

    # 3. Piecewise Linear Transformation (Contrast Stretching)
    piecewise = np.zeros_like(img_gray)
    for i in range(img_gray.shape[0]):
        for j in range(img_gray.shape[1]):
            pixel = img_gray[i, j]
            if pixel < 100:
                piecewise[i, j] = (20 / 100) * pixel
            elif pixel < 200:
                piecewise[i, j] = ((230 - 20) / (200 - 100)) * (pixel - 100) + 20
            else:
                piecewise[i, j] = ((255 - 230) / (255 - 200)) * (pixel - 200) + 230

    composite_bgr = cv2.merge([piecewise, power_trans, log_trans])

    show_images(
        [img_gray, log_trans, power_trans, piecewise, composite_bgr],
        [f'{name} Orig', 'Log (R)', 'Power (G)', 'Piecewise (B)', 'Composite Result'],
        columns=5
    )

Intensity_to_color(lena_gray, "Lena-Grey")
Intensity_to_color(map_img, "Map")
Intensity_to_color(butterfly, "Butterfly")
```





Color Image Enhancements

```
def perform_enhancements(img, name):
    if img is None: return

    neg = 255 - img
    log_t = np.uint8(255/np.log(256) * np.log1p(img.astype(np.float32)))
    gamma = 1.5
    power = np.uint8(np.power(img / 255.0, gamma) * 255)

    smooth = cv2.blur(img, (5, 5))
    kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]])
    sharp = cv2.filter2D(img, -1, kernel)
    tone = cv2.convertScaleAbs(img, alpha=1.2, beta=10)

    show_images([img, neg, log_t, power, smooth, sharp, tone],
                ['Original', 'Negative', 'Log', 'Power', 'Smooth', 'Sharp', 'Tone'])
```

```
perform_enhancements(lena, "Lena")
perform_enhancements(face, "Face")
perform_enhancements(pyramid, "Pyramid")
```